Aerospace Science and Technology 171 (2026) 111625

Contents lists available at ScienceDirect

Aerospace Science and Technology

journal homepage: www.elsevier.com/locate/aescte

Joint multi-target detection and single-target tracking framework for vehicles tracking based on UAV

Jiayu She [a], Yanfang Liu [a,b,c,*], Rui Zhou [a], Naiming Qi [a,b,c]

[a] Department of Aerospace Engineering, Harbin Institute of Technology, Harbin, 150001, China
[b] Suzhou Research Institute of HIT, Suzhou, 215104, China
[c] Zhengzhou Advanced Research Institute of HIT, Zhengzhou, 450000, China

ARTICLE INFO

Communicated by: Yuan Ye

Keywords:
Multiple object tracking
Feature similarity
Feature prediction
Ground vehicle tracking
UAV perspective

ABSTRACT

Currently, most multi-object tracking methods based on Tracking-by-Detection require detecting objects in every frame, demanding substantial computational resources and limiting real-time performance. To address this, we propose a Joint Multi-object Detection and Single-object Tracking (JMDST) framework for ground vehicle tracking from UAV perspective. JMDST consists of two branches: a detection branch and a tracking branch. The detection branch operates periodically to identify all targets in the image at specific intervals, handling targets that dynamically enter or leave the scene. At other times, the tracking branch performs single-object tracking: it predicts target locations, crops small-sized images around these regions, and accurately localizes the targets within these small-sized images to maintain continuous tracking. We propose the Feature Encoding and Location Network (FELNet) to localize targets and extract features within the small-sized images. A dense random sampling strategy and cross-correlation operations are incorporated to enhance the effectiveness of the embedding features. Furthermore, we introduce the Mamba Sequence Feature Prediction (MSFP) module, based on the Mamba network, which predicts the current feature encoding of a target using its historical feature sequence. This enhances robustness to appearance changes and improves trajectory matching accuracy. We evaluate JMDST against state-of-the-art multi-object tracking methods on three UAV-view datasets: MDMT, UAVDT, and VisDrone2019. Experimental results show that JMDST achieves competitive tracking performance with the highest inference speeds on all three datasets, reaching FPS values of 18.9, 26.6, and 18.6, respectively.

1. Introduction

Ground vehicle tracking using UAVs has attracted considerable interest among researchers [1,2], due to its broad range of applications in areas such as traffic monitoring [3], battlefield intelligence gathering [4], and search and rescue operations [5]. Tracking ground vehicles, such as cars, trucks, and buses from a UAV's perspective represents a typical Multiple Object Tracking (MOT) problem. This scenario exhibits several distinctive characteristics compared to other scenarios: ground vehicle targets often appear in small sizes within images; the number of target categories is limited with high inter-class similarity; targets frequently aggregate and disperse; and the background undergoes significant variations under different conditions.

MOT has evolved from traditional methods [6] to the deep learning methods. Benefiting from the significant success of object detection methods, such as Faster-RCNN [7] and YOLO series [8,9], tracking-by-detection [10,11], which means detecting objects on each frame first and then associating them, has become the mainstream in MOT. Tracking-by-detection methods can be categorized into three groups: Separate Detection and Embedding (SDE), Joint Detection and Embedding (JDE), and Joint Detection and Tracking (JDT).

SDE [12] consists of three components: detection, feature embedding, and trajectory association. With the advancement of multi-task learning, JDE [13] employs a one-shot tracker to simultaneously generate object detections and corresponding visual embeddings. For example, STCMOT optimizes the detection and ReID branches simultaneously through a spatio-temporal coupling strategy [14], and QDTrack achieves end-to-end training of object detection and feature encoding using quasi-dense similarity learning [15]. JDT utilizes a single integrated component to perform multi-object tracking. For example, TLSH-MOT uses an end-to-end framework based on Transformer and LSTM to achieve multi-object tracking [16] from the UAV perspective; UTM integrates object detection, feature embedding, and identity association through the IAFE module to form a mutually beneficial positive feedback loop [17]. Although the JDT method has achieved good tracking results, it is difficult to meet the requirements of real-time performance of UAV platforms due to the large amount of computation.

* Corresponding author.
E-mail addresses: shejiayu77@outlook.com (J. She), lyf04025121@126.com (Y. Liu), ruizhou@hit.edu.cn (R. Zhou), qinmok@163.com (N. Qi).

https://doi.org/10.1016/j.ast.2026.111625
Received 23 September 2025; Received in revised form 14 December 2025; Accepted 1 January 2026
Available online 7 January 2026
1270-9638/© 2026 Elsevier Masson SAS. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Nomenclature

$\mathcal{I}_{\mathrm{o}}^{t}$	The input image obtained by UAV
$\mathcal{I}_{\mathrm{s}}^{t}$	The small-sized image cropped from $\mathcal{I}_{\mathrm{o}}^{t}$
w_s, h_s	The width and heigth of cropped SSIs
$\mathcal{T}^{t}$	The trajectory information for all targets at time t
$N_{\mathcal{T}}^{t}$	The number of the targets in $\mathcal{T}^{t}$ at time t
T_{i}^{t}	The tracklet of the i-th target at time t
$\boldsymbol{E}_{\mathcal{T},i}^{t}$	The embedding feature of the target i at time t
$\mathcal{B}_{\mathcal{T},i}^{t}$	The bounding box of the target i at time t
ID_{i}^{t}	The identity of the target i at time t
$\mathcal{S}_{i}^{t}$	The state of the target i at time t
$\mathcal{T}_{\mathrm{c}}^{t}, \mathcal{T}_{\mathrm{uc}}^{t}$	The confirmed and unconfirmed tracklets at time t
$\hat{\boldsymbol{E}}_{\mathcal{T},i}^{t}$	The predicted embedding feature of tracklet i at time t
$\hat{\mathcal{B}}_{\mathcal{T},i}^{t}$	The predicted bounding boxes of tracklets i at time t
N_{T}^{t}	The number of targets tracked by tracking module
τ	The detection interval
D^{t}	The detection results in $\mathcal{I}_{\mathrm{o}}^{t}$
N_{D}^{t}	The number of the detected targets in $\mathcal{I}_{\mathrm{o}}^{t}$ at time t
$\{\mathcal{B}_{j}^{t}\}_{j=1}^{N_{D}^{t}}$	The bounding boxes of the detected targets at time t
$\{\boldsymbol{E}_{j}^{t}\}_{j=1}^{N_{D}^{t}}$	The embedding features of detected targets at time t
Training Process	
k	The sampling interval for video during training
$k_{\max}$	The max sampling interval during training
N_{o}	The number of images per batch during training
N_{s}	The number of SSIs cropped from an $\mathcal{I}_{\mathrm{o}}^{t}$ during training
$\bar{\boldsymbol{o}}_n, \hat{\boldsymbol{o}}_n$	The ground-truth and predicted overlap vector by FELNet during training
$\bar{\boldsymbol{C}}_n, \hat{\boldsymbol{C}}_n$	The ground-truth and predicted confidence by FELNet during training
$\hat{\boldsymbol{E}}_{i}^{t}$	The features encoded by FELNet for target i at time t during training
$\mathcal{L}_{\hat{\boldsymbol{o}}}, \mathcal{L}_{\hat{\boldsymbol{C}}}, \mathcal{L}_{\hat{\boldsymbol{E}}}$	The loss function for $\boldsymbol{o}$, $\boldsymbol{C}$, and $\boldsymbol{E}$
$\lambda_1, \lambda_2, \lambda_3$	The weight of $\mathcal{L}_{\hat{\boldsymbol{o}}}$, $\mathcal{L}_{\hat{\boldsymbol{C}}}$, and $\mathcal{L}_{\hat{\boldsymbol{E}}}$
$N_{\mathrm{P}}, N_{\mathrm{N}}, N_{\mathrm{pair}}$	The number of positive, negative, and all pairs for $\hat{\boldsymbol{E}}$ during training
$T_{\mathcal{FCC}}(i,j)$	Whether $\hat{\boldsymbol{E}}_{i}^{t_a}$ and $\hat{\boldsymbol{E}}_{j}^{t_b}$ correspond to the same target
$\bar{\boldsymbol{E}}_{i_k}^{t-m}$	The ground-truth feature of object i_k at $t-m$ frame
$\boldsymbol{M}$	The length of the feature sequence used in MSFP
N_{b}	The batchsize of feature sequences for training MSFP
$\mathcal{L}_{\mathrm{manba}}$	The loss function for MSFP
Metrics	
$\mathcal{T}_{\mathrm{c,m}}^{t-1}$, D_{m}^{t}	The matched tracklets and detections in cascade matching
$\mathcal{T}_{\mathrm{c,um}}^{t-1}$	The unmatched tracklets in cascade matching
D_{um}^{t}	The unmatched detections in cascade matching
$\mathcal{T}_{\mathrm{m}'}^{t-1}$, $D_{\mathrm{m}'}^{t}$	The matched tracklets and detections in IoU assignment
$\mathcal{T}_{\mathrm{um}'}^{t-1}$	The unmatched tracklets in IoU assignment
$D_{\mathrm{um}'}^{t}$	The unmatched detections in IoU assignment
N	Number of frames for the tracklet to transition from uncomfirm state to comfirm state
c_1, i_1, i_2	Unmatched trajectory determination threshold
c_t	The number of correctly matches between the predicted trajectory and the ground truth at time t
$\mathrm{IoU}_{t,i}$	The IoU of the i th matching pair at time t
MOTA	Multiple Object Tracking Accuracy
MOTP	Multiple Object Tracking Precision
IDs	ID Switching
IDF1	Identification F_1 Score
DetA	Detection Accuracy
AssA	Association Accuracy
HOTA	Higher Order Tracking Accuracy
TP, FP, FN, and GT	The number of true positives, false positives, false negatives, and ground truth
TPA, FNA, and FPA	The number of true positive, false negative, and false positive associations
IDTP, IDFP, and IDFN	The number of the true positive, false positive, and false negative trajectories

Trajectory association methods also significantly influence the overall tracking performance, and can be categorized into three types: position-based matching, feature-based matching, and joint matching methods. Position-based matching methods [10] associate trajectories by leveraging the spatial proximity (e.g., Intersection over Union (IoU) or center distance) of objects between consecutive frames. In complex scenarios, object motion is often nonlinear [18]. Motion estimation (Multi-Bernoulli smoother [19], unscented Kalman filter [20], or other nonlinear filters [21]) or displacement regression [22–24] can be used to enhance the measurement of spatial proximity for the same object. However, when objects occlude each other and exhibit similar motion patterns, methods based solely on position often struggle to effectively distinguish between different objects. Feature-based matching methods [25], inspired by the human ability to recognize objects via appearance, aim to learn discriminative features for different objects and then associate them based on feature similarity. Nevertheless, in scenarios such as traffic monitoring, different targets often exhibit high appearance similarity, making it challenging for existing appearance-based methods to distinguish between them. Joint matching methods combine the advantages of both approaches: spatial proximity primarily helps reduce mismatches among distant objects with similar appearances, while feature similarity usually plays a supporting role in reinforcing correct associations or re-identifying lost targets [15].

Despite advances, existing approaches often overlook several critical aspects: (1) Most methods typically require object detection in

every frame, posing a significant challenge for real-time processing. (2) For ground vehicle tracking, only a limited set of attributes is necessary to distinguish different objects. However, the embedding features used in existing methods often have high channel dimensionality, resulting in redundant information. (3) The embedding features of an object are susceptible to variations in shooting distance, angle, illumination, and other environmental conditions.

It is noteworthy that single-object tracking methods typically achieve high processing speeds because they focus on only one target. For instance, the lightweight CTDT method proposed in Ref. [26] can reach speeds exceeding 100 FPS; benefiting from the batch computation capabilities of CNN, it also supports parallel processing of multiple single-object trackers. This inspires us to explore a new framework that combines periodic detection with lightweight single-object tracking: performing global detection periodically to capture scene changes and new targets, while using fast single-object tracking on intermediate frames to maintain trajectory continuity.

However, effectively integrating the two approaches to simultaneously satisfy the requirements of high accuracy and high speed still faces the following challenges:

(1) Periodic detection and continuous tracking coordination: The core challenge lies in integrating two alternating processes. Specifically, in detection frames, the framework must correctly associate detections with existing trajectories and appropriately select tracking targets for non-detection frames. In non-detection frames, single-object trackers must accurately inherit and continue the target tracking state to maintain trajectory continuity.

(2) The robust feature representation problem: It is challenging to learn compact yet highly discriminative embedding features that are both sufficient to distinguish between visually similar ground vehicles and remain robust to environmental perturbations such as scale, viewpoint variations, and lighting conditions.

(3) Temporal feature consistency: Target appearance features naturally evolve over time, causing a discrepancy between past and current representations. Relying solely on direct feature matching can thus lead to association errors. A mechanism is needed to predict or smooth the current appearance based on historical feature sequences, thereby reducing mismatches and identity switches caused by temporal appearance changes.

To address the aforementioned issues, we propose a Joint Multi-target Detection and Single-target Tracking (JMDST) framework for ground vehicle tracking using UAVs. The main contributions of this work are summarized as follows:

(1) We propose the JMDST framework, which integrates two branches: a detection branch and a tracking branch. The detection branch operates periodically to identify all targets. Between two consecutive detections, the tracking branch crops the small-sized images (SSIs) of each target based on the detection results and maintains individual tracking and feature encoding for each target, thereby improving the overall processing speed. This hybrid approach balances the need for comprehensive scene understanding with computational efficiency.

(2) We introduce the Feature Encoding and Location Network (FELNet), designed for precise position prediction and feature encoding of objects from SSIs. To eliminate redundant information in embedding features, FELNet employs a lightweight architecture with reduced channel counts and outputs compact, low-dimensional feature vectors specifically tailored for ground vehicle distinction. A dense random sampling strategy and cross-correlation operations are incorporated to enhance feature robustness and distinctiveness.

(3) We introduce the Mamba Sequence Feature Prediction (MSFP) module, based on the Mamba network [27]. To mitigate feature sensitivity to variations in shooting distance, angle, illumination, and other environmental conditions, the MSFP module predicts object features at the current frame based on historical appearance sequences, thereby adapting to target appearance variations over time and reducing identity switches. By leveraging temporal consistency, MSFP provides more stable feature representations that are robust to transient environmental changes.

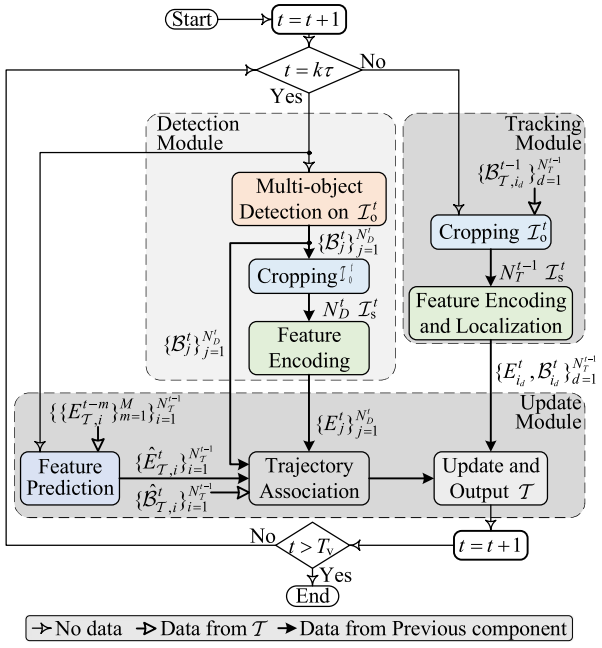


Fig. 1. The overall framework of the proposed JMDST method.

The rest of this paper in organized as follows. Section 2 details the proposed JMDST framework, including FELNet, MSFP, trajectory association and update. Section 3 presents experiments on three UAV datasets, with ablation studies and comparisons to state-of-the-art methods. Finally, Section 4 concludes the paper and outlines future work.

2. Method

2.1. Overview

At time t, the trajectory information for all targets is defined as $\mathcal{T}^t = \{\mathrm{T}_i^t\}_{i=1}^{N_\mathcal{T}^t}$, where T_i^t denotes the tracklet of the i-th target. Each tracklet T_i^t comprises the following components: the sequence of historical features of the target: $\{\boldsymbol{E}_{\mathcal{T},i}^{t-m}\}_{m=0}^{t}$, its bounding box at time t: $\mathcal{B}_{\mathcal{T},i}^t$, its identity: ID_i^t, and its state at time t, denoted as $\mathcal{S}_i^t$, which can be: tentative state, confirmed state, and deleted state.

The JMDST method consists of three modules: a detection module, a tracking module, and an update module, as illustrated in Fig. 1. Specifically, the detection module identifies all existing targets in the image. The tracking module maintains continuous tracking of multiple targets that are in a confirmed state and for which the time since the last update is below a predefined threshold. The update module matches newly detected targets with existing trajectories and updates the trajectories using association results derived from the matching process or directly from the tracking module. The detection and tracking modules are executed intermittently. When $t = k\tau$, where $k = 0, 1, 2, \ldots$, the detection module is activated, where τ denotes the detection interval. At all other time steps, the tracking module is executed.

The detection module encompasses multi-object detection, cropping, and feature encoding. When $t = k\tau$, multi-object detection is performed on the input image $\mathcal{I}_o^t$ to locate all potential targets, yielding N_D^t detected targets along with their bounding boxes $\{\mathcal{B}_j^t\}_{j=1}^{N_D^t}$, N_D^t is the number of detected targets in $\mathcal{I}_o^t$. Based on these detections, N_D^t SSIs $\mathcal{I}_s^t$ are cropped and passed through FELNet to generate feature embeddings $\{\boldsymbol{E}_j^t\}_{j=1}^{N_D^t}$ for each target. Meanwhile, the predicted feature em-

beddings $\{\hat{\boldsymbol{E}}^t_{\mathcal{T},i}\}_{i=1}^{N^{t-1}_{\mathcal{T}}}$ for the $N^{t-1}_{\mathcal{T}}$ existing targets are predicted using their historical feature sequences, where $N^{t-1}_{\mathcal{T}}$ is the number of targets in $\mathcal{T}^t$. Similarly, their predicted bounding boxes $\{\hat{\mathcal{B}}^t_{\mathcal{T},i}\}_{d=1}^{N^{t-1}_{\mathcal{T}}}$ are predicted via a Kalman filter. The N^t_D detected targets are then associated with the $N^{t-1}_{\mathcal{T}}$ existing trajectories using both feature similarity (comparing $\{\hat{\boldsymbol{E}}^t_{\mathcal{T},i}\}_{i=1}^{N^{t-1}_{\mathcal{T}}}$ and $\{\boldsymbol{E}^t_j\}_{j=1}^{N^t_D}$) and spatial consistency (comparing $\{\hat{\mathcal{B}}^t_{\mathcal{T},i}\}_{i=1}^{N^{t-1}_{\mathcal{T}}}$ and $\{\mathcal{B}^t_j\}_{j=1}^{N^t_D}$). Based on the association results, the trajectory set $\mathcal{T}^t$ is updated, yielding N^t_T targets $\{\mathrm{T}^t_{i_d}\}_{d=1}^{N^t_T} \in \mathcal{T}^t$ to be tracked over the subsequent $\tau - 1$ frames.

The tracking module consists of three main steps: cropping, feature encoding, and localization. First, the possible positions of the N^{t-1}_T targets in $\{\mathrm{T}^{t-1}_{i_d}\}_{d=1}^{N^{t-1}_T}$ at time t are predicted, where N^{t-1}_T is the number of targets tracked by tracking module at time t. Then, N^{t-1}_T SSIs are cropped according to the predicted bounding boxes $\{\hat{\mathcal{B}}^t_{\mathcal{T},i_d}\}_{d=1}^{N^{t-1}_T}$. These SSIs are processed by FELNet to extract feature embeddings $\{\boldsymbol{E}^t_{i_d}\}_{d=1}^{N^{t-1}_T}$ and refine the positions $\{\mathcal{B}^t_{\mathcal{T},i_d}\}_{d=1}^{N^{t-1}_T}$ of the N^{t-1}_T targets within the SSIs. Finally, the trajectory information $\mathcal{T}^t$ is updated and output. The JMDST framework application routine is outlined in Algorithm 1.

Algorithm 1 JMDST framework application routine.

Input: Video frame sequence $\mathcal{I}^t_{\mathrm{o}}$, detection interval τ, YOLOv11, FELNet, MSFP.
Output: Confirm-state tracklets updated at time t.

Initialize $\mathcal{T}^0 = \emptyset$, $t = 0$.
for each frame $\mathcal{I}^t_{\mathrm{o}}$ **do**
  if $t = k\tau, k = 0, 1, 2, \ldots$ **then**
    YOLOv11 performs object detection on $\mathcal{I}^t_{\mathrm{o}}$ and obtains the positions $\{\mathcal{B}^t_j\}_{j=1}^{N^t_D}$ of N^t_D targets.
    Crop N^t_D SSIs based on $\{\mathcal{B}^t_j\}_{j=1}^{N^t_D}$.
    Input SSIs into FELNet to obtain the targets' encoded features $\{\boldsymbol{E}^t_j\}_{j=1}^{N^t_D}$.
    For each tracklet $T^{t-1}_i \in \mathcal{T}^{t-1}$, predict $\hat{\boldsymbol{E}}^t_{\mathcal{T},i}$ using Eq. (8).
    For each tracklet $T^{t-1}_i \in \mathcal{T}^{t-1}$, predict $\hat{\mathcal{B}}^t_{\mathcal{T},i}$ using Kalman Filter.
    Perform cascade matching between $\mathcal{T}^{t-1}_{\mathrm{c}}$ and D^t based on $\hat{\boldsymbol{E}}^t_{\mathcal{T},i}$ and $\{\boldsymbol{E}^t_j\}_{j=1}^{N^t_D}$.
    Perform IoU assignment based on $\mathcal{T}^{t-1}_{\mathrm{uc}}$, $\mathcal{T}^{t-1}_{\mathrm{c,um}}$, and D^t_{um}.
    Update $\mathcal{T}^t$ and Kalman filter.
  else
    For each tracklet $T^{t-1}_{i_d}$, predict $\hat{\mathcal{B}}^t_{\mathcal{T},i_d}$ using Kalman Filter.
    Crop N^{t-1}_T SSIs based on $\{\hat{\mathcal{B}}^t_{\mathcal{T},i_d}\}_{d=1}^{N^{t-1}_T}$.
    Input SSIs into FELNet to obtain encoded features $\{\boldsymbol{E}^t_{i_d}\}_{d=1}^{N^{t-1}_T}$ and overlap vectors $\{\boldsymbol{o}^t_{i_d}\}_{d=1}^{N^{t-1}_T}$.
    Use Eq. (2) to convert $\{\boldsymbol{o}^t_{i_d}\}_{d=1}^{N^{t-1}_T}$ into $\{\mathcal{B}^t_{i_d}\}_{d=1}^{N^{t-1}_T}$.
    Update $\mathcal{T}^t$ and Kalman Filter.
  end if
  Output confirm-state tracklets which are updated at time t in $\mathcal{T}^t$.
  $t = t + 1$.
end for

2.2. Feature encoding and localization network

In the tracking module, the original image $\mathcal{I}^t_{\mathrm{o}}$ is cropped into N^{t-1}_T SSIs $\mathcal{I}^t_{\mathrm{s}}$, each centered around the corresponding bounding box in $\{\hat{\mathcal{B}}^t_{\mathcal{T},i_d}\}_{d=1}^{N^{t-1}_T}$. Each SSI has a fixed size of $w_{\mathrm{s}} \times h_{\mathrm{s}} = 64 \times 64$. As illustrated in Fig. 2, FELNet takes $\mathcal{I}^t_{\mathrm{s}}$ as input and outputs the following for each target: an encoded feature vector $\boldsymbol{E}$, a confidence score $\boldsymbol{C}$, and an overlap vector $\boldsymbol{o}$ representing the overlap between the target and the SSI [26]. Given the bbox $\mathcal{B}_{\mathrm{S}} = [l_1, u_1, w_1, h_1]^{\mathrm{T}}$ of $\mathcal{I}^t_{\mathrm{s}}$ and the bbox $\mathcal{B}_{\mathrm{H}} = [l_2, u_2, w_2, h_2]^{\mathrm{T}}$ of the object relative to $\mathcal{I}^t_{\mathrm{o}}$, the overlap vector is defined as $\boldsymbol{o}(\mathcal{B}_{\mathrm{S}}, \mathcal{B}_{\mathrm{H}}) = [o_1, o_2, o_3, o_3]^{\mathrm{T}}$, where o_1 and o_2 are the width overlap length, and o_3 and o_4 are the height overlap length. They are given as [28]:

$$\begin{cases} o_1 = \max(0, l_1 + w_1 - l_2) \\ o_2 = \max(0, l_2 + w_2 - l_1) \\ o_3 = \max(0, u_1 + h_1 - u_2) \\ o_4 = \max(0, u_2 + h_2 - u_1) \end{cases} \tag{1}$$

Given $\mathcal{B}_{\mathrm{S}} = [l_1, u_1, w_1, h_1]^{\mathrm{T}}, \boldsymbol{o}(\mathcal{B}_{\mathrm{S}}, \mathcal{B}_{\mathrm{H}}) = [o_1, o_2, o_3, o_3]^{\mathrm{T}}$, $\mathcal{B}_{\mathrm{H}} = [l_2, u_2, w_2, h_2]^{\mathrm{T}}$ can be calculated as [28]:

$$\begin{cases} l_2 = l_1 + w_1 - o_1 \\ u_2 = u_1 + h_1 - o_3 \\ w_2 = o_1 + o_2 - w_1 \\ h_2 = o_3 + o_4 - h_1 \end{cases} \tag{2}$$

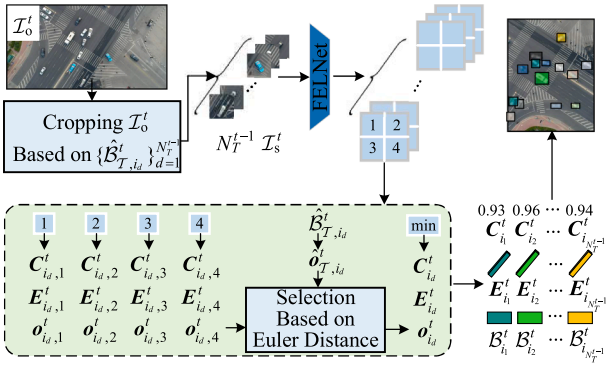


Fig. 2. The framework of feature encoding and localization.

FELNet is an improved architecture based on Darknet-53 [29], as outlined in Table 1. The design of FELNet incorporates several key modifications: First, since the target occupies limited information within each cropped SSI, the network does not require convolutional kernels with a large number of channels to extract features. Accordingly, we reduce the channel counts throughout the Darknet-53 backbone. Additionally, the depth of the network (i.e., the number of stacked layers) is correspondingly decreased. Moreover, the original fully connected layer is replaced with three separate branches, each implemented in a fully convolutional manner. These branches independently output the overlap vector $\boldsymbol{o}$, the embedding feature, and the confidence score. Thanks to these structural changes, FELNet achieves fast image processing with a lightweight architecture.

FELNet partitions the input image $\mathcal{I}^t_{\mathrm{s}}$ into a grid of four anchor boxes, each covering a region of size 32×32 within the image. If the ground-truth bounding box of a target contains the center point of an anchor box, that anchor box is assigned to localize and embed the features of the target. During training, if multiple targets cover the center point of the same anchor box, the loss is computed using the label of the target with the smallest deviation in overlap vector. Anchor boxes that do not overlap with any target are assigned an overlap vector and a confidence label of 0.

When training FELNet, the sampling interval k is randomly chosen from $[1, k_{\max}]$, and N_{o} original images $\mathcal{I}_{\mathrm{o}}$ are selected at intervals of k along the video sequence. Here, $k_{\max}$ is adjustable based on the apparent motion speed of targets in the image. From each selected $\mathcal{I}_{\mathrm{o}}$, N_{s} SSIs $\mathcal{I}^t_{\mathrm{s}}$

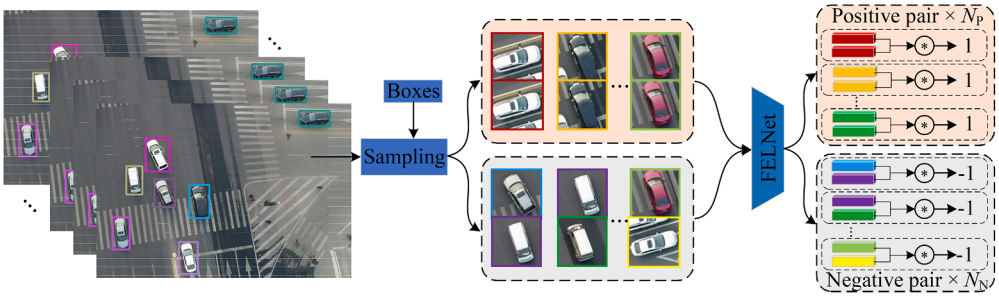


Fig. 3. Feature similarity learning for FELNet network.

are randomly cropped base on the object labels. To improve robustness to detection inaccuracies, each I_s^t is augmented via shifting, flipping, and scaling. The loss function for o is as follows:

$$\mathcal{L}_{\hat{o}} = \sqrt{\frac{\sum_{n=0}^{N_o N_s} \sum_{a=1}^{4} (\bar{o}_{n,a} - \hat{o}_{n,a})^2}{4 N_o N_s}} \tag{3}$$

where $\bar{o}_n$ denotes the true overlap vector and $\hat{o}_n$ represents the prediction of FELNet, $a = 1, 2, 3, 4$ indexing the four anchor boxes. The confidence loss associated with the overlap vectors is defined as follows:

$$\mathcal{L}_{\hat{C}} = \sqrt{\frac{\sum_{n=0}^{N_o N_s} \sum_{a=1}^{4} (\bar{C}_{n,a} - \hat{C}_{n,a})^2}{4 N_o N_s}} \tag{4}$$

where $\bar{C}_{n,a}$ denotes the ground-truth confidence label for anchor box a, which equals 1 if the anchor box corresponds to a target, and 0 otherwise. $\hat{C}_{n,a}$ represents the corresponding confidence prediction from FELNet.

The $N_o \times N_s$ SSIs I_s are also used to learn the feature similarity, as illustrated in Fig. 3. Let $\hat{E}_i^{t_a}$ and $\hat{E}_j^{t_b}$ denote the features encoded by FELNet for target i at time t_a and target j at time t_b, respectively. If $i = j$, then $\hat{E}_i^{t_a}$ and $\hat{E}_j^{t_b}$ form a positive feature pair; otherwise, they form a negative feature pair. From the FELNet embedding features of targets across all $N_o \times N_s$ SSIs I_s, we sample N_P positive pairs and N_N negative pairs. A fast cross-correlation function $\mathcal{FCC}()$ is used to measure feature similarity. The corresponding feature loss is defined as follows:

$$\mathcal{L}_{\hat{E}} = \sqrt{\frac{\sum (\mathcal{FCC}(\hat{E}_i^{t_a}, \hat{E}_j^{t_b}) - T_{\mathcal{FCC}}(i,j))^2}{N_{\text{pair}}}} \tag{5}$$

where $N_{\text{pair}} = N_P + N_N$ is the total number of feature pairs, $T_{\mathcal{FCC}}(i,j)$ is defined as:

$$T_{\mathcal{FCC}}(i,j) = \begin{cases} 1, i = j \\ -1, i \neq j \end{cases} \tag{6}$$

The FELNet is jointly optimized under

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\hat{E}} + \lambda_2 \mathcal{L}_{\hat{o}} + \lambda_3 \mathcal{L}_{\hat{C}} \tag{7}$$

where λ_1, λ_2 and λ_3 are the weight of $\mathcal{L}_{\hat{E}}$, $\mathcal{L}_{\hat{o}}$, $\mathcal{L}_{\hat{C}}$ and set to 1.0, 1.0 and 1.0 by default in this paper. The training routine of FELNet is outlined in Algorithm 2

FELNet produces four sets of outputs. The set that best represents the target's current state (including position, feature, and confidence) is selected as follows. The selection process is as follows: first, the predicted bounding box $\hat{\mathcal{B}}_{\mathcal{T},id}^{t-1}$ is converted into an overlap vector relative to the cropped image. Then, the Euclidean distance is computed between this overlap vector and each of the four output overlap vectors predicted by FELNet. Among the output sets with confidence greater than 0.9, the one with the smallest Euclidean distance is selected.

2.3. Mamba sequence feature prediction module

Although storing feature embeddings from previous frames provides a direct method to obtaining tracklet features, these embeddings can become unreliable due to occlusion and identity switches. Furthermore, the appearance of a target may change over time as it moves or as the camera perspective shifts, leading to variations in its feature embeddings across frames. Existing methods for sequence feature prediction primarily employ CNNs, RNNs, or Transformers. While Transformers address the limitations of CNNs and RNNs in capturing long-range dependencies, they suffer from high computational complexity. Recently, Mamba has been introduced [27], known for its superior efficiency, reduced GPU memory consumption, and improved long-distance dependency modeling, showing good results in sequence prediction tasks. To improve the robustness of tracklet features and achieve more efficient object association, we propose the MSFP module. The MSFP leverages Mamba to predict the current feature embedding based on the historical feature

Table 1
Detailed structure of FELNet network.

	Type	Channel Number	Kennel Size/Stride	Output Size
	Convolutional	16	3 × 3/1	64 × 64
	Convolutional	32	3 × 3/2	32 × 32
	Convolutional	16	3 × 3/1	
1×	Convolutional	32	3 × 3/1	
	Residual			32 × 32
	Convolutional	64	3 × 3/2	16 × 16
	Convolutional	32	3 × 3/1	
2×	Convolutional	64	3 × 3/1	
	Residual			16 × 16
	Convolutional	128	3 × 3/2	8 × 8
	Convolutional	64	3 × 3/1	
6×	Convolutional	128	3 × 3/1	
	Residual			8 × 8
	Convolutional	128	3 × 3/2	4 × 4
	Convolutional	128	3 × 3/1	
6×	Convolutional	128	3 × 3/1	
	Residual			4 × 4
	Convolutional	128	3 × 3/2	2 × 2
	Convolutional	128	3 × 3/1	
4×	Convolutional	128	3 × 3/1	
	Residual			2 × 2
	Convolutional	64	1 × 1/1	2 × 2
$o \leftarrow$	Convolutional	32	1 × 1/1	2 × 2
	Convolutional	16	1 × 1/1	2 × 2
	Convolutional	4	1 × 1/1	2 × 2
	Convolutional	64	1 × 1/1	2 × 2
$E \leftarrow$	Convolutional	32	1 × 1/1	2 × 2
	Convolutional	16	1 × 1/1	2 × 2
	Convolutional	16	1 × 1/1	2 × 2
	Convolutional	64	1 × 1/1	2 × 2
$C \leftarrow$	Convolutional	32	1 × 1/1	2 × 2
	Convolutional	16	1 × 1/1	2 × 2
	Convolutional	1	1 × 1/1	2 × 2

Algorithm 2 Training routine of FELNet.

Input: FELNet, $k_{\max}$, N_o, $\{\mathcal{I}_o\}$, N_s, N_P, N_N.
Output: Trained FELNet.

```
Initialize FELNet.
while Training epoches have not been completed do
    Choose randomly k from [1, k_max].
    Randomly select N_o I_o from {I_o} at intervals of k.
    From each selected I_o, randomly crop N_s SSIs I_s^t according to the object labels.
    Augmente I_s through shifting, flipping, and scaling.
    Feed I_s into FELNet, and obtain {Ê_j, ô_j, Ĉ_j}.
    Use Eq. (1) to convert object labels to {ō_j}.
    Sample N_P positive and N_N negative pairs from {Ê_j}.
    Calculate the feature similarity labels for N pairs of features using Eq. (6).
    Calculate the total loss function based on Eq. (3), (4), (5), (7).
    Calculate gradients and update the parameters of FELNet.
end while
```

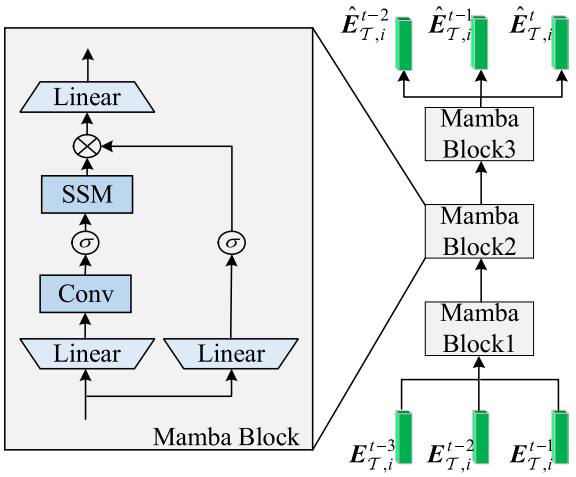


Fig. 4. Detailed structure of sequence feature prediction module.

sequence of a target. The process can be formulated as:

$$\{\hat{\boldsymbol{E}}^{t}_{\mathcal{T},i}, \cdots, \hat{\boldsymbol{E}}^{t-M}_{\mathcal{T},i}\} = \mathrm{MSFP}(\{\boldsymbol{E}^{t-1}_{\mathcal{T},i}, \cdots, \boldsymbol{E}^{t-M-1}_{\mathcal{T},i}\}) \tag{8}$$

where $\boldsymbol{E}^{t}_{\mathcal{T},i}$ denotes the feature embedding of target i at time t, $\hat{\boldsymbol{E}}^{t}_{\mathcal{T},i}$ is the predict feature embedding, M is the length of the feature sequence used in MSFP. As illustrated in Fig. 4, the MSFP module comprises three Mamba blocks stacked in sequence, each sharing the same structure.

During training, the loss function is defined as:

$$\mathcal{L}_{\mathrm{mamba}} = \sqrt{\frac{\sum_{k=0}^{N_b}\sum_{m=0}^{M}(\bar{\boldsymbol{E}}^{t-m}_{i_k} - \hat{\boldsymbol{E}}^{t-m}_{i_k})^2}{M N_b}} \tag{9}$$

where $\bar{\boldsymbol{E}}^{t-m}_{i_k}$ is the ground-truth feature of object i_k at $t-m$ frame, and $\hat{\boldsymbol{E}}^{t-m}_{i_k}$ is the predict feature of object i_k at time $t-m$, N_b is the number of object feature sequences in a batch.

2.4. Trajectory association and update

In this paper, trajectory association is improved on the basis of DeepSORT. At time t, given the history trajectory information $\mathcal{T}^{t-1} = \{\mathrm{T}^{t-1}_i\}_{i=1}^{N^{t-1}_{\mathcal{T}}}$, the prediction features and bounding boxes are denoted as $\{\hat{\boldsymbol{E}}^{t}_{\mathcal{T},i}\}_{i=1}^{N^{t-1}_{\mathcal{T}}}$ and $\{\hat{\mathcal{B}}^{t}_{\mathcal{T},i}\}_{i=1}^{N^{t-1}_{\mathcal{T}}}$, respectively. The trajectories in $\mathcal{T}^{t-1}$ are categorized into confirmed tracklets $\mathcal{T}^{t-1}_{\mathrm{c}}$ and unconfirmed tracklets $\mathcal{T}^{t-1}_{\mathrm{uc}}$, where $\mathcal{T}^{t-1} = \mathcal{T}^{t-1}_{\mathrm{c}} + \mathcal{T}^{t-1}_{\mathrm{uc}}$. When a new target appears in the image, the detection module first identifies it and initializes a new unconfirmed tracklet. This tracklet is then continuously tracked by the tracking module over the subsequent $\tau - 1$ frames. An unconfirmed tracklet is promoted to a confirmed state only after being consistently detected in N consecutive frames. If $N \le \tau$, the tracklet may be confirmed during the tracking phase; however, false positives generated by the detection or tracking modules would also be output as confirmed targets, thereby increasing the false positive rate. Conversely, if $N > \tau + 1$, even a tracklet successfully detected in two consecutive detection cycles would remain unconfirmed, leading to a rise in false negatives. To balance false positives and false negatives, we set $N = \tau + 1$.

At time $t = k\tau$, multi-object detection is performed and features encoding is applied to the detected objects. A total of N^t_D objects are detected in $\mathcal{I}^t_o$, denoted as $D^t = \{D^t_j\}_{j=1}^{N^t_D}$. Their corresponding bounding boxes and embedding features are denoted as $\{\mathcal{B}^t_j\}_{j=1}^{N^t_D}$ and $\{\boldsymbol{E}^t_j\}_{j=1}^{N^t_D}$. The purpose of trajectory association is to match D^t_i with its corresponding tracklet $\mathcal{T}^{t-1}_i$ belonging to the same object.

First, cascade matching is performed between $\mathcal{T}^{t-1}_{\mathrm{c}}$ and D^t based on embedding features. The matching results fall into three categories: matched tracklets and detections ($\mathcal{T}^{t-1}_{\mathrm{c,m}}$, D^t_{m}), unmatched tracklets ($\mathcal{T}^{t-1}_{\mathrm{c,um}}$), and unmatched detections (D^t_{um}), where $\mathcal{T}^{t-1}_{\mathrm{c}} = \mathcal{T}^{t-1}_{\mathrm{c,m}} + \mathcal{T}^{t-1}_{\mathrm{c,um}}$ and $D^t = D^t_{\mathrm{m}} + D^t_{\mathrm{um}}$. Next, the IoU is computed between the following sets' bounding boxes: $\mathcal{T}^{t-1}_{\mathrm{uc}}$, $\mathcal{T}^{t-1}_{\mathrm{c,um}}$ (time since the update is no larger than $\tau + 1$), and D^t_{um} to obtain the cost matrix. Then, IoU assignment is applied based on this cost matrix, yielding three outcomes: matched tracklets and detections ($\mathcal{T}^{t-1}_{\mathrm{m}'}$, $D^t_{\mathrm{m}'}$), unmatched tracklets ($\mathcal{T}^{t-1}_{\mathrm{um}'}$), and unmatched detections ($D^t_{\mathrm{um}'}$). Matched tracklets ($\mathcal{T}^{t-1}_{\mathrm{c,m}}$ and $\mathcal{T}^{t-1}_{\mathrm{m}'}$) are updated using the Kalman filter. Unmatched unconfirmed tracklets are deleted immediately, while unmatched confirmed tracklets are deleted after 100 consecutive mismatches. $D^t_{\mathrm{um}'}$ are utilized to initialize new tracklets.

Due to the periodic execution of the detection module, relying solely on single-target tracking for detected objects could result in missed targets being lost for up to τ consecutive frames in case of detection failures. To mitigate this, we expand the set of tracking targets. At time $t \neq k\tau$, feature encoding and localization are performed for not only the targets detected by the detection module at time $t = k\tau$, but also the targets from unmatched tracklets $\mathcal{T}^{t-1}_{\mathrm{um}'}$ that meet the following criteria: they are in a confirmed state, the time since their last update is no greater than $\tau + 1$, and their distance from the image boundary is greater than d. This strategy allows us to maintain tracking of missed targets using previously stored information, even when they are not detected in the most recent cycle. Furthermore, targets near the image border are excluded because they may have already moved outside the field of view.

The tracking results originating from unmatched trajectories also undergo filtering based on the following criteria: (1) Targets are considered false positives if their confidence is below threshold c_1, or if the IoU between their bounding box and any detection bounding box exceeds threshold i_1. (2) If the IoU between two targets from unmatched trajectories exceeds threshold i_2, only the target with higher confidence is retained. The filtered results from the tracking module are then used to update the corresponding trajectories.

2.5. Training

The JMDST framework includes three trainable modules: FELNet, MSFP, and the object detector. In this work, we use YOLOv11 as the detector. YOLOv11 is trained separately using its default parameters. Meanwhile, FELNet is trained, independently, using the Adam optimizer. This is necessary because MSFP relies on the embedding features generated by FELNet. To accelerate convergence and avoid local minima, a cosine annealing learning rate schedule is employed. After FELNet converges, MSFP is trained with the Adam optimizer and the same cosine annealing strategy, using the root mean square error (RMSE) as the loss function.

3. Experiments

In this section, we evaluate JMDST on three benchmark datasets: MDMT [30], UAVDT [31], and VisDrone2019 [32]. We demonstrate its effectiveness through comparisons with state-of-the-art approaches and conduct extensive ablation studies to assess each component of our framework.

3.1. Datasets

MDMT: MDMT is a multi-UAV multi-target tracking dataset, captured simultaneously by two UAVs. In other words, each video sequence consists of paired recordings. The dataset is divided into a training set (24 sequences), a validation set (5 sequences), and a test set (13 sequences). MDMT includes three object categories: person, bicycle, and car. We consider only the car category. Additionally, we use only half of the available paired sequences.

UAVDT: The UAVDT dataset is designed for ground vehicle detection and tracking from UAV perspectives. It contains three object categories: cars, trucks, and buses. For the MOT task, the dataset includes 50 video sequences. In this work, we focus solely on the car category. Accordingly, we select 23 sequences from UAVDT, which are partitioned into 12 for training, 3 for validation, and 8 for testing. The video frames have a resolution of 1080×540 pixels and cover diverse common scenarios such as squares, arterial roads, and toll stations.

VisDrone2019: The VisDrone2019 dataset is captured using UAVs across 14 different cities in China, covering both urban and rural environments. It contains 12 object categories: ignore area, pedestrian, person, bicycle, car, van, truck, tricycle, awning-tricycle, bus, motor, and others. For evaluation, we consider only four ground vehicle categories: car, van, truck, and bus. Accordingly, we select 45 sequences for training, 10 for validation, and 10 for testing from the VisDrone2019 dataset.

3.2. Metrics

To evaluate JMDST against other state-of-the-art approaches, we employ the following widely-used metrics: Multiple Object Tracking Accuracy (MOTA), Multiple Object Tracking Precision (MOTP), ID Switching (IDs) and Identification F_1 Score (IDF1), Detection Accuracy (DetA), Association Accuracy (AssA), Higher Order Tracking Accuracy (HOTA).

MOTA provides a comprehensive evaluation of both detection and tracking accuracy. It is defined as follows [33]:

$$\mathrm{MOTA} = 1 - \frac{\mathrm{FP} + \mathrm{FN} + \mathrm{IDs}}{\mathrm{GT}} \tag{10}$$

where FP, FN, and GT are the number of false positives, false negatives, and ground truth, IDs is the number of identity switches.

MOTP measures the localization precision of the tracker and is defined as [33]:

$$\mathrm{MOTP} = \frac{\sum_{t,i} \mathrm{IoU}_{t,i}}{\sum_t c_t} \tag{11}$$

where c_t is the number of correctly matches between the predicted trajectory and the ground truth at time t, $\mathrm{IoU}_{t,i}$ represents the IoU of the i th matching pair at time t.

IDF1 evaluates association performance by matching ground truth and predicted trajectories at the track level and computing the corresponding F1-score. It is defined as [34]:

$$\mathrm{IDF1} = \frac{2\mathrm{IDTP}}{2\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}} \tag{12}$$

Where IDTP, IDFP, and IDFN are the true positive, false positive, and false negative trajectories.

HOTA is the geometric mean of DetA and AssA and is defined as [35]:

Table 2
Ablation study about some components of JMDST-3 on the MDMT.

Method	MOTA↑	MOTP↑	IDs↓	IDF1↑	HOTA↑	FN↓	FP↓
YOLOv11 + FE	67.9	**78.8**	361	79.0	62.6	41,269	13509
YOLOv11 + ReID	67.6	78.3	541	79.2	61.8	42,935	12146
YOLOv11 + FE + MSFP	**68.3**	**78.8**	**335**	**79.6**	**62.9**	**40696**	13306

$$\mathrm{HOTA} = \sqrt{\mathrm{DetA} \times \mathrm{AssA}} \tag{13}$$

$$\mathrm{DetA} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN} + \mathrm{FP}} \tag{14}$$

$$\mathrm{AssA} = \frac{1}{\mathrm{TP}} \sum_{c \in \mathrm{TP}} \frac{\mathrm{TPA}(c)}{\mathrm{TPA}(c) + \mathrm{FNA}(c) + \mathrm{FPA}(c)} \tag{15}$$

where TP is the true positive detection, TPA, FNA, and FPA are the true positive, false negative, and false positive associations.

3.3. Implementation details

SSIs are cropped from the original image based on the predicted bounding box of the target at the current frame. Rather than cropping directly using the bounding box coordinates, the crop is centered at the bounding box center, with a width and height of $\sqrt{(1.3w + 0.3h)(1.3h + 0.3w)}$, and then resized to 64×64. The size of the cropped region is dynamically determined to define a search area that robustly encloses the target. This design follows two guiding principles: (1) the cropping region is expanded proportionally to the target's width and height, which incorporates contextual information and provides a tracking margin; (2) the square root of the product of the dimensions is used to compute an effective average, preventing the crop size from being dominated by either dimension alone and thus accommodating targets with varied aspect ratios. The choice of this specific input size is grounded in experimental findings from prior research. As demonstrated in Ref. [36], a resolution of 64×64 represents the optimal input size for the target scale distribution present in the datasets used in this study. This configuration effectively balances localization accuracy and computational efficiency, which is essential for the demands of our target tracking task. If the target has a highly asymmetric aspect ratio, the method described above may cause the target to fall near the centers of multiple anchor boxes without fully overlapping any one of them. To mitigate this, the crop dimensions are instead set to $\min(1.9w, 1.9h, \sqrt{(1.3w + 0.3h)(1.3h + 0.3w)})$. Furthermore, to account for potential localization inaccuracies, especially when the target bounding box only marginally covers an anchor box center, the cropping range is randomly translated by 16 pixels. This encourages better alignment between the target and the anchor box centers.

The proposed method is implemented using Pytorch on an RTX3070 GPU. We employ YOLOv11 as the detector and apply Non-maximum Suppression (NMS) with an IoU threshold of 0.2 and a confidence threshold of 0.55. In the update module, when processing results from the tracking branch, the following parameter values are used: $d = 5$, $c_1 = 0.9$, $i_1 = 0.1$, $i_2 = 0.1$.

3.4. Ablation study

To validate effectiveness of the JMDST framework, ablation experiments are conducted to analyze the proposed components on the MDMT dataset.

3.4.1. Effectiveness of MSFP

To evaluate the effectiveness of the MSFP module, we exclude the features predicted by MSFP when associating existing trajectories with

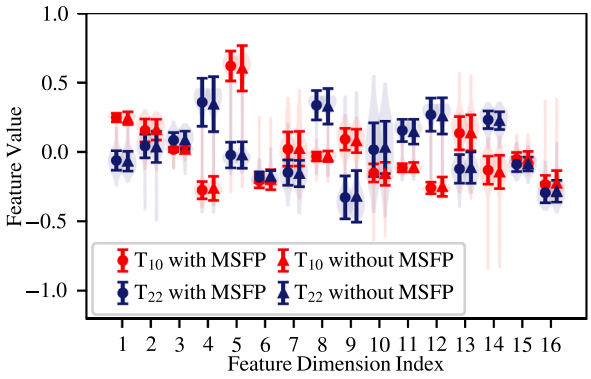


Fig. 5. Visualization of Features for T_{10} and T_{22} from MDMT-26-1.

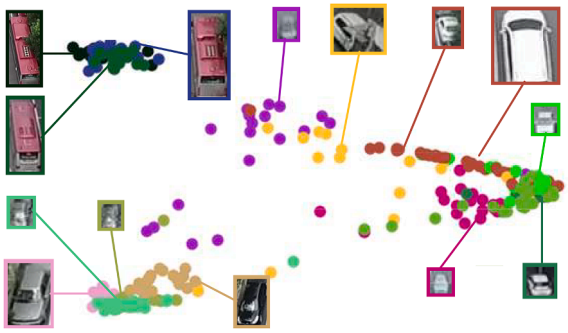


Fig. 6. Feature embedding space visualization for targets from MDMT-26-1.

detection results. Instead, we use the most recently updated features of the targets for each existing trajectory. The results of JMDST without MSFP are presented in Table 2.

JMDST without MSFP achieves a MOTA of 67.9, MOTP of 78.8, IDs of 361, IDF1 of 79.0, and HOTA of 62.6. With the incorporation of MSFP, MOTA increases slightly by 0.59%, IDF1 by 0.76%, and HOTA by 0.32%. Additionally, the number of IDS decreases from 361 to 335. These results indicate that MSFP primarily enhances the matching between detection results and existing trajectories. We visualize the feature distribution of target ID 10 in video sequence 26-1, as shown in Fig. 5. When sequence features are not used for prediction, the target's feature distribution is relatively dispersed. This dispersion arises from continuous variations in viewpoint and position, combined with inherent noise in feature embedding, leading to high variance and instability in representing the target. In contrast, when MSFP is employed, the feature distribution becomes more compact, indicating that the predicted features offer a more stable and consistent representation. Our experimental results confirm that using predicted features improves the accuracy of detection-to-trajectory matching.

3.4.2. *Effectiveness of feature encoding of FELNet*

To evaluate the effectiveness of the features from the feature encoding branch of FELNet in trajectory association, we replaced this branch with the deep appearance descriptor from DeepSORT [12]-hereafter referred to as ReIDNet-and measured feature similarity using cosine distance based on its encoded features. We selected ReIDNet as the baseline for two primary reasons. First, since the trajectory association in JMDST builds upon the DeepSORT framework, employing the same ReID network enables a direct and methodologically consistent comparison. Second, the use of deep ReID features for association, initially established by DeepSORT, has become a widely adopted benchmark in multi-object tracking [37,38], and its effectiveness continues to be affirmed and refined in later works [39]. This offers a strong and relevant baseline, ensuring a meaningful evaluation of our proposed feature encoding approach. The results are presented in the second row of Table 2. It can be observed that, compared to ReIDNet, the feature encoding from FELNet leads to slight improvements in MOTA, MOTP, IDF1, HOTA, FN, and FP. More notably, FELNet reduces the number of IDs significantly from 541 to 335, a decrease of 38.1%, indicating its enhanced capability in preserving identity consistency during association. Furthermore, due to the lower-dimensional feature representation obtained through FELNet, the average processing time per cropped image is reduced to 0.33 ms, compared to 0.40 ms with ReIDNet, demonstrating a clear advantage in computational efficiency. Fig. 6 illustrates the feature distributions of various targets. Targets with similar appearance exhibit smaller feature distances, while those with distinct appearances are farther apart in the feature space, confirming the discriminative power of the learned embeddings.

Table 3
Ablation study about interval for activation detection of JMDST on the MDMT.

Method	MOTA↑	MOTP↑	IDs↓	IDF1↑	HOTA↑	FN↓	FP↓	FPS↑
JMDST-2	**69.0**	**79.2**	377	**79.8**	**63.8**	**38417**	14,416	15.9
JMDST-3	68.3	78.8	335	79.6	62.9	40,696	13,306	18.9
JMDST-4	65.8	78.7	341	78.0	61.4	44,882	13,491	19.3
JMDST-5	65.0	78.6	253	78.1	60.9	46,965	12,896	19.9
JMDST-6	63.4	78.7	**208**	77.0	60.1	50,788	**11747**	**20.3**

3.4.3. *Effectiveness of detection module's frequency*

To investigate the impact of executing the detection branch once every τ frames and the tracking branch at all other times, we compared the tracking performance and speed of the algorithm under different values of τ. The results are summarized in Table 3. As τ increases, MOTA, MOTP, IDF1, and HOTA gradually decrease, indicating a reduction in tracking accuracy. This decline occurs primarily because longer detection intervals can cause the algorithm to miss newly appearing targets during non-detection frames, leading to an increased number of missed detections. This is reflected in the rise of FN. Interestingly, as τ increases, both IDs and FP decrease. This can be attributed to two factors: Single-target tracking achieves more stable identity maintenance than position and feature-based matching, thereby reducing IDs; Only targets that persist for $\tau + 1$ consecutive frames are promoted to the confirmed state, which helps filter out false detections and reduces FP. Meanwhile, FPS of the algorithm continues to rise with larger τ, though the rate of increase diminishes gradually. Considering both accuracy and efficiency, $\tau = 3$ offers the most balanced trade-off.

3.4.4. *Effectiveness of detector*

The performance of the detector considerably influences the tracking accuracy of the algorithm. To evaluate the impact of the detector on JMDST, we replaced YOLOv11 with YOLOX. As shown in Table 4, using YOLOX resulted in decreases of 8.6% in MOTA, 1.1% in MOTP, 6.0% in IDF1, and 19.3% in HOTA, along with a reduction in FP. In contrast, IDs and FN increased by 32.5% and 31.2%, respectively. With the exception of FP, all other metrics are worse than achieved with YOLOv11. Although most metrics deteriorated compared to using YOLOv11, the results underscore a critical point: even with a weaker detector, the JMDST framework maintains a functional and competitive tracking performance. This robustness can be attributed to the framework's dual-branch design, which mitigates the impact of intermittent detection failures through its dedicated single-target tracking branch. The tracking branch sustains trajectories based on historical data and motion prediction, thereby reducing the dependency on perfect per-frame detection. This design choice is pivotal for real-world UAV applications where detector performance may vary due to environmental conditions or platform constraints.

Based on the above analysis, in subsequent comparisons, unless otherwise specified, we will use YOLOv11 as the detector under the condition of $\tau = 3$ to compare with existing methods.

Table 4
Overall performance comparison with other tracking algorithms on the MDMT. The best two performances are displayed in red and blue.

Methods	MOTA↑	MOTP↑	IDs↓	IDF1↑	HOTA↑	DetA↑	AssA↑	FP↓	FN↓	FPS↑
DeepSORT	61.0	78.9	514	75.1	60.4	51.9	71.0	13,933	52,438	9.2
CTracker	40.5(-20.5)	**80.0(+1.1)**	3773	34.5(-40.6)	33.4(-27.0)	38.5(-13.4)	29.2(-41.8)	11,804	86,719	1.9(-7.3)
SORT	59.7(-1.3)	**79.9(+1.0)**	642	74.4(-0.7)	61.0(+0.6)	52.7(+0.8)	71.5(+0.5)	14,797	53,758	16.5(+7.3)
ByteTrack	62.0(+1.0)	78.7(-0.2)	373	76.1(+1.0)	61.1(+0.7)	52.4(+0.5)	72.0(+1.0)	13,432	51,435	17.2(+8.0)
DiffMOT	64.2(+3.2)	79.7(+0.8)	899	75.6(+0.5)	62.0(+1.6)	53.7(+1.8)	**72.5(+1.5)**	10,372	50,221	14.9(+5.7)
DiffusionTrack	62.3(+1.3)	79.4(+0.5)	**113**	77.4(+2.3)	61.7(+1.3)	52.3(+0.4)	**73.4(+2.4)**	**9140**	54,579	9.3(+0.1)
QDTrack	**68.7(+7.7)**	77.6(-1.3)	**195**	**81.1(+6.0)**	**62.4(+2.0)**	**54.5(+2.6)**	72.0(+1.0)	**7339**	**46235**	10.5(+1.3)
JMDST	**68.3(+7.3)**	78.8(-0.1)	335	**79.6(+4.5)**	**62.9(+2.5)**	**57.2(+5.3)**	69.7(-1.3)	13,306	**40696**	**18.9(+9.7)**
JMDST + YOLOX	62.4(+1.4)	78.1(-0.8)	444	74.8(-0.3)	58.6(-1.8)	52.0(+0.1)	66.6(-1.3)	10,741	50,788	**18.8(+9.6)**

Table 5
Overall performance comparison with other tracking algorithms on the UAVDT.

Methods	MOTA↑	MOTP↑	IDs↓	IDF1↑	HOTA↑	DetA↑	AssA↑	FP↓	FN↓	FPS↑
DeepSORT	31.4	75.6	200	48.3	38.4	31.3	47.7	9090	45,489	15.8
CTracker	43.3(+11.9)	80.7(+5.1)	1356	36.3(-12.0)	33.7(-4.7)	41.3(+10.0)	27.6(-20.1)	9825	34,162	3.9(-11.9)
SORT	30.2(-1.2)	75.7(+0.1)	**161**	46.9(-1.4)	37.0(-1.4)	29.2(-2.1)	47.6(-0.1)	**7470**	48,139	**25.9(+10.1)**
ByteTrack	32.0(+0.6)	74.9(-0.7)	165	49.8(+1.5)	39.2(+0.8)	31.6(+0.3)	49.2(+1.5)	9539	44,670	22.9(+7.1)
DiffMOT	35.9(+4.5)	75.8(+0.2)	452	42.6(-5.7)	36.4(-2.0)	35.6(+4.3)	37.9(-9.8)	11,099	39,533	19.3(+3.5)
DiffusionTrack	46.9(+15.5)	75.3(-0.3)	**129**	62.3(+14.0)	46.5(+8.1)	40.7(+9.4)	53.5(+5.8)	**8753**	33,577	9.6(-6.2)
QDTrack	**69.7(+38.3)**	**81.5(+5.9)**	179	**76.5(+28.2)**	**62.9(+24.5)**	**59.2(+27.9)**	**67.0(+19.3)**	9165	**14817**	13.0(-2.8)
JMDST	**67.3(+35.9)**	**81.2(+5.6)**	170	**73.5(+25.2)**	**60.1(+21.7)**	**59.0(+27.7)**	**61.3(+13.6)**	10,791	**15155**	**26.6(+10.8)**

3.5. Comparison to state-of-the-arts

To extensively evaluate the proposed method, we compare our JMDST with CTracker [24], SORT [10], DeepSORT [12], ByteTrack [40], DiffMOT [41], DiffusionTrack [42], and QDTrack [15] on the aforementioned UAV MOT benchmarks, i.e., MDMT, UAVDT and VisDrone2019.

3.5.1. MDMT

As shown in Table 4, the proposed JMDST demonstrates superior performance compared to other state-of-the-art methods across several key metrics. Our method achieves the best results in HOTA, DetA, FN, and FPS on this benchmark. Although JMDST does not outperform QDTrack in MOTA, MOTP, IDs, or IDF1, it exhibits significantly higher inference speed. QDTrack attains stronger tracking performance at the cost of a more complex network architecture and higher matching computational overhead, resulting in a processing rate of only 10.5 FPS. In contrast, our method runs at 18.9 FPS, owing primarily to the dual-branch design that integrates joint multi-object detection and single-object tracking.

Furthermore, although YOLOX is inferior to YOLOv11 in detection performance, JMDST equipped with YOLOX still achieves a MOTA of 62.4 and an IDF1 of 74.8, outperforming most YOLOX-based methods such as SORT, DeepSORT, ByteTrack, and DiffusionTrack. Meanwhile, JMDST equipped with YOLOX maintains an inference speed of 18.8 FPS, surpassing all comparison methods. This further highlights the framework's adaptability and stability in practical deployment.

3.5.2. UAVDT

Table 5 presents a comparison between JMDST and existing MOT methods on the UAVDT test set. This dataset exhibits considerable divergence between training and test scenarios, as well as significant differences in target size distribution. Under these challenging conditions, most metrics of QDTrack are slightly higher than those of JMDST. However, due to a high number of missed detections, the performances of all other methods are substantially lower than both JMDST and QDTrack. This outcome can be attributed to two factors: (1) The powerful detection capability of YOLOv11 helps accurately identify targets, greatly reducing the number of FN. (2) Even when the detector misses certain targets intermittently, the incorporation of single-object tracking helps maintain trajectories of these targets, further decreasing FN. These results demonstrate that our method can effectively alleviate the impact of detector missed detections on multi-object tracking. Moreover, JMDST achieves a processing rate of 26.6 FPS on UAVDT, outperforming all other methods, and achieving twice the speed of QDTrack. Overall, the experimental outcomes confirm the strong and efficient performance of our JMDST framework.

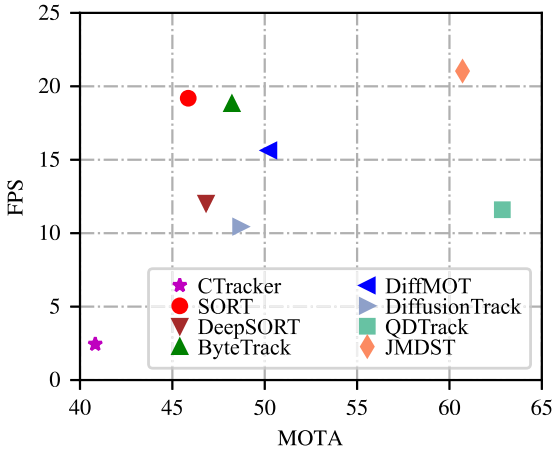


Fig. 7. Average MOTA-FPS comparisons of different trackers on MDMT, UAVDT and VisDrone2019.

3.5.3. VisDrone2019

The scenes in VisDrone2019 are more complex, leading to further degradation in detection performance. The results on the VisDrone2019 test set are consistent with those on UAVDT: JMDST performs slightly worse than QDTrack but surpasses all other methods. Moreover, JMDST achieves the highest average speed, operating 1.6 times faster than QDTrack. These findings further highlight the characteristics of JMDST, demonstrating that it offers strong tracking performance while maintaining superior real-time capability.

Fig. 7 compares the average MOTA and FPS of JMDST against other trackers across three datasets. JMDST attains the highest average FPS, even exceeding SORT and ByteTrack which rely solely on positional in-

Table 6
Overall performance comparison with other tracking algorithms on the VisDrone2019.

Methods	MOTA↑	MOTP↑	IDs↓	IDF1↑	HOTA↑	DetA↑	AssA↑	FP↓	FN↓	FPS↑
DeepSORT	36.3	75.8	905	47.9	38.7	34.3	44.3	12,984	60711	12.2
CTracker	39.6(+3.3)	81.1(-4.9)	3021	35.6(-12.3)	32.7(-6.0)	37.1(+2.8)	29.0(-15.3)	5990	61,756	2.1(-10.1)
SORT	36.1(-0.2)	76.2(+0.4)	944	47.2(-0.7)	38.4(-0.3)	33.2(-1.1)	45.2(+0.9)	10,328	63,587	16.8(+4.6)
ByteTrack	39.0(+2.7)	75.6(-0.2)	457	53.6(+5.7)	41.5(+2.8)	34.9(+0.6)	50.2(+5.9)	10,561	60,467	**16.9(+4.7)**
DiffMOT	39.3(+3.0)	76.3(+0.5)	694	52.8(+4.9)	42.1(+3.4)	36.1(+1.8)	49.7(+5.4)	12,098	58,345	13.4(+1.2)
DiffusionTrack	40.5(+4.2)	75.3(-0.5)	652	55.0(+7.1)	43.2(+4.5)	37.9(+3.6)	50.1(+5.8)	15,416	**53631**	13.2(+1.0)
QDTrack	**49.5(+13.2)**	**77.2(+1.4)**	**161**	**66.0(+18.1)**	**49.0(+10.3)**	**41.0(+6.7)**	**58.9(+14.6)**	**7798**	**51185**	11.3(-0.9)
JMDST	**44.9(+8.6)**	**76.5(+0.7)**	**178**	**62.4(+14.5)**	**46.4(+7.7)**	**38.0(+3.7)**	**56.9(+12.6)**	9293	55,074	**18.6(+6.4)**

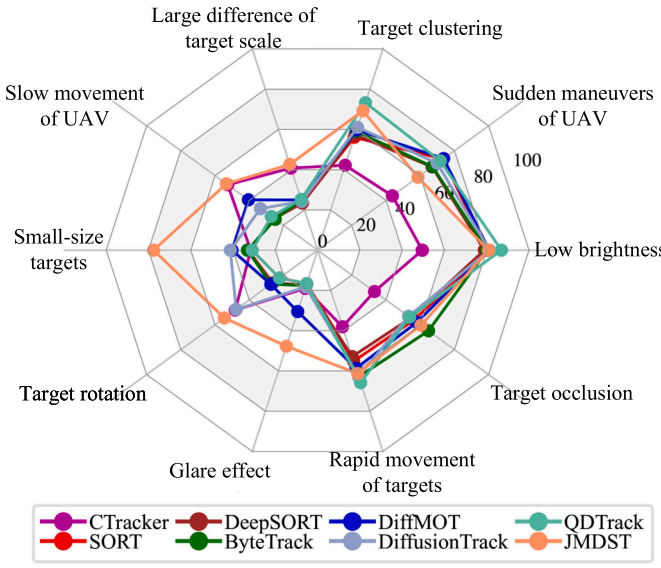


Fig. 8. Tracking performance comparison of JMDST with other tracking algorithms under various challenging attributes, evaluated using MOTA..

formation for matching, and its MOTA significantly outperforms both SORT and ByteTrack. Although the average MOTA of JMDST is slightly lower than that of QDTrack, it runs approximately twice as fast. This indicates that JMDST achieves a favorable balance between tracking accuracy and real-time processing efficiency.

3.5.4. Attribute-based evaluation and qualitative results

To provide a deeper understanding of JMDST's performance characteristics and directly substantiate the strengths and limitations of its architecture, we conduct a detailed attribute-based analysis complemented by qualitative visualizations. The attribute-based performance analysis involving 10 distinct attributes: low brightness (MDMT-57-1), sudden maneuvers of UAV (MDMT-48-1), target clustering (MDMT-68-1), large different of target scale (MDMT-26-1), slow movement of UAV (UAVDT-M0203), small size targets (UAVDT-M0402), target rotation (UAVDT-M0901), glare effect (UAVDT-1008), rapid movement of targets (VisDrone2019-uav0000249_00001_v), target occlusion (VisDrone2019-uav0000297_00000_v).

Fig. 8 presents a comparative evaluation of JMDST against other methods under these attributes. In scenarios involving large difference of target scale, slow movement of UAV, small-size targets, target rotation, and glare effect, JMDST consistently outperforms other methods. Fig. 9 presents qualitative results of JMDST in these scenarios. The good results can be attributed to several key design aspects: The SSI cropping strategy adaptively ensures the relative scale of the target, improving localization accuracy across targets of different sizes. FELNet is specifically designed to extract discriminative features from SSIs, enhancing robustness to small targets and rotations. The MSFP module utilizes historical feature sequences to predict current features, thereby stabilizing representations under slow motion or gradually changing appearances (such as rotation and glare).

Conversely, JMDST exhibits slightly lower performance in rapid movement of targets, low brightness, and target clustering, where it is surpassed by QDTrack, and in target occlusion, where ByteTrack performs better.

Low brightness and similar target clustering can challenge FELNet's feature discrimination capabilities. As shown in Fig. 10(a), the detector initially missed a target in Frame 548 due to tree obstruction, but successfully detected it in Frame 550. Due to the similar appearance and adjacent locations between targets, ID-97 was incorrectly assigned to the target during match, while the origin target was assigned a new ID, resulting in an IDs. Subsequently, due to occlusion, the target was lost again at frame 552, and ID-97 was reassigned back to the original target, resulting in another IDs. Low brightness can lead to missed detection, simultaneously weakening FELNet's feature discriminability. In Fig. 10(b), the target appears completely dark, causing the detection method to miss the target in most cases. Even when detected, FELNet's subsequent tracking performance is suboptimal. In such situations, the performance of JMDST is still only slightly inferior to QDTrack.

Occlusion will cause the target state to be unavailable for a period of time. As shown in Fig. 10(c), the target with ID-24 reappears after being occluded by the overpass for a long time, and relying only on the tracking branch cannot fully restore its ID. The algorithm assigns a new ID to it.

The rapid movement of the target can cause the target to be displaced in the tracking frame beyond the predicted range of the Kalman filter, resulting in an incorrect SSI cropping position and an increase in IDs, as shown in Fig. 10(d). It is worth noting that during sudden maneuver of UAV, the changes in the position of multiple targets may exceed the prediction ability of the motion model, resulting in SSI cropping and tracking failure, and the cropping area may partially contain other targets or even no targets at all, resulting in IDs, FP, and FN, as shown in Fig. 10(e). One potential solution is to incorporate real-time UAV motion information [43]. By integrating the UAV's self-motion with the target's motion model, the prediction of the target position can be improved, resulting in more accurate image cropping and reduced tracking failures.

3.5.5. Computational complexity and scalability

Computational efficiency is critical for deploying tracking systems in large-scale real-world drone swarms. QDTrack represents one of the most accurate trackers on UAV benchmarks. A direct comparison between JMDST and QDTrack (as shown in Table 7) is crucial to understand the practical trade-offs. QDTrack relies on per-frame detection of a heavy backbone network and global similarity matching, leading to higher per-frame computational load and precision. JMDST addresses the real-time requirement through a dual-branch architecture that interleaves periodic, computation-intensive global detection with continuous, low-cost single-object tracking. Furthermore, the framework employs a lightweight FELNet to process 64×64 SSIs and extract a compact 16-dimensional feature representation, together with efficient Mamba blocks for feature-sequence prediction. Compared to QDTrack's stan-

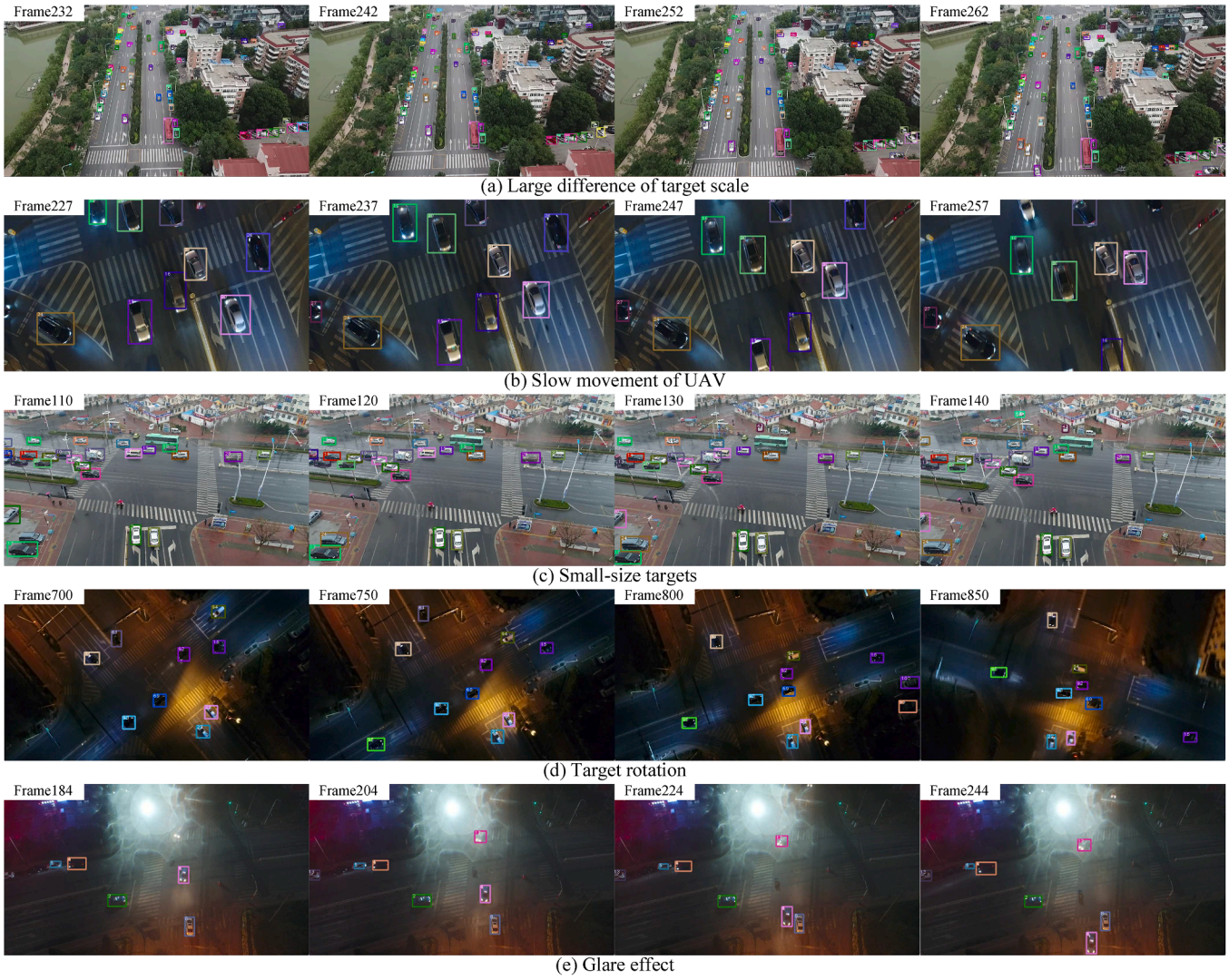


Fig. 9. Qualitative results of JMDST.

Table 7
Architectural and efficiency comparison between JMDST and QDTrack based on MDMT.

Aspect	JMDST	QDTrack
Detection strategy	Periodic	Every frame
Feature extractor	FELNet	ResNet-50 + FPN
Feature predictor	MSFP	–
Feature length	16	512
Tracklet association	Cascade match and IoU match	Bi-directional softmax
FPS (RTX3070)	18.9	10.5
FPS (Jetson Orin NX)	5.0	2.0
GPU memory usage	1.8 GB	2.2 GB
System memory usage	3.5 GB	4.3 GB

dard high-dimensional features from ResNet-50 and RPN, this design drastically reduces memory and computation overhead during tracklet association. To validate the practicality on a real UAV platform, we deployed JMDST and QDTrack on the NVIDIA Jetson Orin NX module. JMDST maintained a processing speed of over 5 frames per second on the MDMT dataset, demonstrating its suitability for real-time embedded deployment. In contrast, the heavier computational load of QDTrack makes it difficult to achieve similar speeds on such resource-constrained platform.

Additionally, the computational load of JMDST scales positively with the number of targets in the scene and inversely with the detection interval τ. Consequently, τ can be adapted according to scene-specific demands: in scenarios with few, slowly-varying targets and stringent efficiency requirements, a larger τ can improve algorithm speed; whereas in dynamic environments with complex target motions and relaxed real-time constraints, a smaller τ can enhance tracking accuracy. In large-scale operations involving high-resolution imagery, our approach offers an additional advantage. Conventional frame-by-frame detectors typically downsample the full image to a fixed input size, which reduces the pixel-level representation of targets and can degrade detection performance. In contrast, the JMDST tracking module crops a region adapted to each target's extent, then resizes it to a consistent dimension before feeding it into FELNet-thereby preserving the target's relative scale. This strategy grants the tracking module greater robustness to variations in image resolution.

3.5.6. *Robustness to noise and image degradation*

To further assess the impact of background noise and image degradation on tracking robustness, we simulate four common types of image-quality deterioration on the MDMT dataset: Gaussian noise (mean 0, standard deviation 10), condensation, dirty of camera, and icing, as illustrated in Fig. 11. The tracking performance of JMDST under these

Table 8
Performance of each method after degradation of image quality.

Methods	Mode of Simulation	MOTA↑	IDF1↑	Mode of Simulation	MOTA↑	IDF1↑
DeepSORT	Gaussian Noise	60.5	74.5	Dirty of camera	59.7	74.2
CTracker		28.5	28.9		38.7	33.9
SORT		58.3	73.8		57.7	73.3
ByteTrack		61.4	75.5		60.6	75.0
DiffMOT		55.1	68.1		62.5	74.1
DiffusionTrack		61.7	76.4		59.9	74.7
QDTrack		**66.3**	**78.9**		**66.8**	**79.4**
JMDST		**64.3**	**76.7**		**65.7**	**76.9**
deepsort	Condensation	60.3	74.0	Icing	60.0	74.1
Ctracker		36.7	32.6		30.2	31.4
SORT		58.1	72.8		57.6	73.7
ByteTrack		61.2	75.4		60.8	75.2
DiffMOT		53.2	64.6		62.6	74.1
DiffusionTrack		61.1	75.9		60.9	75.4
QDTrack		**65.9**	**78.9**		**66.6**	**79.4**
JMDST		**65.6**	**76.6**		**65.8**	**76.6**

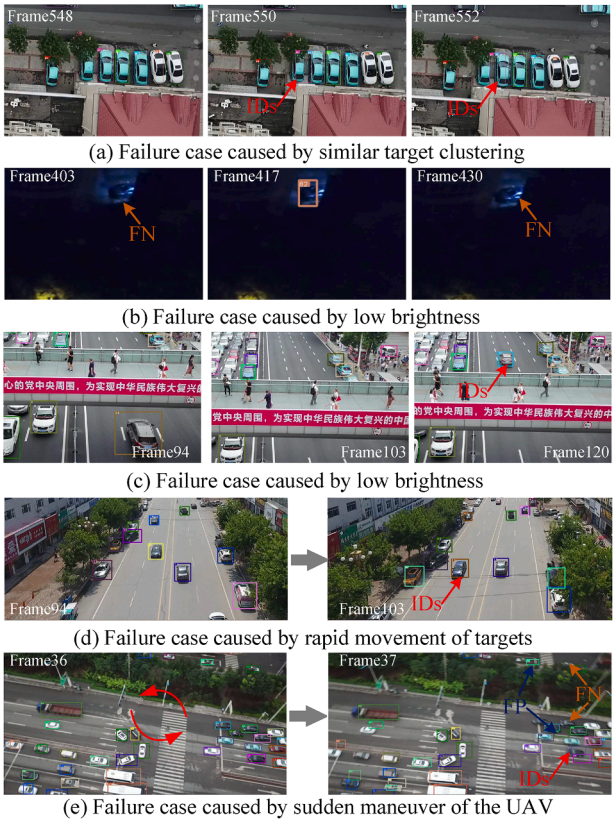


Fig. 10. Examples of failure cases.

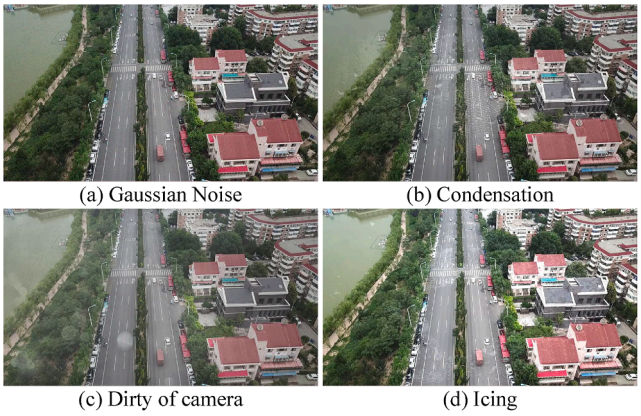


Fig. 11. Simulated effect with degradation of image quality.

degradations, measured by MOTA and IDF1 and compared with other state-of-the-art methods, is summarized in Table 8.

Experimental results show that JMDST maintains robust performance across all degradation types. Under pixel-wise Gaussian noise, JMDST achieves a MOTA of 64.3 and an IDF1 of 76.7, ranking second only to QDTrack while surpassing all other methods. More severe degradations, such as dirty of camera, icing, and condensation, simulate extreme UAV operational environments, introducing local irregular occlusions or extensive blurring. Even under these conditions, JMDST exhibits only a minor performance drop, remaining competitive with the top-performing QDTrack and outperforming other benchmarks. This demonstrates its strong resilience to local occlusion and blurring effects.

The robustness of JMDST can be attributed to its dual-branch architecture. While the periodic detection module may be affected by noise in a given frame, the tracking branch leverages historical data and motion predictions from earlier frames to preserve trajectory continuity, thereby mitigating transient disturbances. The MSFP module plays a key role by predicting current target features from historical feature sequences, smoothing feature variations and maintaining identity consistency even when the target's appearance is temporarily corrupted by noise, occlusion, or blur. In contrast, methods that rely on per-frame detection and appearance matching, such as CTracker and DiffMOT, suffer notable performance declines under degradation because detection or feature-extraction errors propagate directly into the association stage. QDTrack, which employs dense similarity learning, also shows strong robustness, albeit at substantially higher computational cost.

These results confirm that JMDST not only strikes a favorable balance between accuracy and speed, but also sustains reliable tracking performance under common real-world image degradations. This combination of efficiency and robustness makes the framework well-suited for practical UAV deployment in diverse and challenging environmental conditions.

3.6. Discussion on performance trade-offs

As shown in Tables 4–6, JMDST does not achieve the lowest IDs or FP on the MDMT and UAVDT datasets. This outcome reflects a deliberate trade-off inherent in our framework, which prioritizes real-time performance. Unlike conventional tracking methods that execute a detector on every frame, JMDST employs a hybrid architecture that interleaves periodic detection with continuous single-target tracking. This design is the principal reason for its superior FPS rate. However, operating without

detection in certain frames introduces error sources that directly affect ID consistency and FP:

(1) In the tracking branch, the SSI is cropped based on a Kalman-filter prediction and the target's previous state. When a target undergoes abrupt motion or is occluded, prediction errors can accumulate, leading to an SSI that does not fully contain the target. Upon re-detection, this positional misalignment can cause feature matching to fail, resulting in an identity switch.

(2) Severe prediction errors may shift the estimated bounding box far from the true target location, causing the corresponding SSI to capture background or other objects. If the network subsequently yields a high-confidence output for such a region, a persistent false-positive trajectory can be initiated. Due to the continuous nature of the tracking branch, these false positives may endure over several frames before being discarded, thereby increasing the FP count.

In contrast, methods such as QDTrack perform high-precision detection and feature matching in every frame, achieving higher per-frame association accuracy and thus better ID and FP metrics-at the cost of significantly greater computational overhead. By accepting a modest compromise in IDs and FP, JMDST attains a substantial gain in FPS while maintaining strong performance across other metrics. This balance is particularly valuable for real-time UAV applications where throughput is critical. Notably, on the more complex and densely crowded VisDrone2019 dataset, JMDST exhibits moderately better ID and FP performance compared to its results on MDMT and UAVDT, underscoring its effectiveness in challenging scenarios. This also indicates that on relatively simpler datasets, the observed differences in certain metrics stem primarily from the architectural emphasis of the framework rather than from fundamental limitations of the method itself.

4. Conclusion

In this article, we propose a multi-object tracking framework called JMDST, which integrates multi-object detection and single-object tracking to achieve real-time ground vehicle tracking UAV-based. The framework performs multi-object detection at regular intervals to identify all existing targets and adapt to targets dynamically entering or exiting the field of view. Between detection steps, SSIs are cropped based on the predicted target positions, and single-object tracking is applied to maintain continuous trajectories, significantly improving computational efficiency. We introduce FELNet, a network designed to accurately localize targets and extract discriminative features from SSIs. By incorporating a dense random sampling strategy and cross-correlation operations, FELNet enhances the robustness and distinctiveness of feature encoding. Furthermore, we propose the MSFP module based on the Mamba network, which leverages historical feature sequences to predict current feature encoding, thereby improving trajectory association accuracy. We evaluate JMDST on three UAV-based tracking datasets: MDMT, UAVDT, and VisDrone2019, comparing it against state-of-the-art MOT methods. Experimental results show that JMDST achieves the highest frame rates (18.9, 26.6, and 18.6 FPS, respectively) on these datasets while maintaining competitive tracking accuracy. Specifically, it attains MOTA scores of 68.3 on MDMT, 67.3 on UAVDT, and 44.9 on VisDrone2019. In addition, attribute-based evaluation demonstrates that JMDST outperforms other methods in challenging scenarios including large scale variations, slow UAV motion, small target size, target rotation, and glare effects.

While the proposed JMDST framework demonstrates competitive performance and efficiency, it is not without limitations, which also point to valuable directions for future research. First, the framework employs a fixed detection interval. While this design is key to its efficiency, it may result in delayed output when new targets appear after a detection frame, or sustained error propagation when tracking branches fail during a long interval. Second, despite the robustness introduced by MSFP, tracking accuracy can still degrade under extreme conditions such as persistent occlusion, very abrupt and fast camera motion, or when a target undergoes drastic appearance change within a single detection interval. Third, the current training pipeline involves separate training of the detector, FELNet, and MSFP. To address these issues, in the future we will study adaptive control mechanisms for detection intervals. By integrating additional information, such as IMU data, to stabilize predictions during drone maneuvers, robustness can be enhanced. Finally, an end-to-end jointly optimized system might learn more synergistic representations, potentially improving overall performance.

CRediT authorship contribution statement

Jiayu She: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Data curation, Conceptualization; **Yanfang Liu:** Writing – review & editing, Supervision, Software, Resources, Project administration, Investigation, Funding acquisition, Formal analysis, Conceptualization; **Rui Zhou:** Writing – review & editing, Supervision, Software, Project administration, Formal analysis; **Naiming Qi:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Data curation.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by National Key Research and Development Program of China (2022YFB3902701), National Natural Science Foundation of China (52272390), and National High-Level Young Scholars Program (Q2022335).

References

[1] M. Betke, E. Haritaoglu, L.S. Davis, Real-time multiple vehicle detection and tracking from a moving vehicle, Mach. Vision Appl. 12 (2) (2000) 69–83.
[2] W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu, T.-K. Kim, Multiple object tracking: a literature review, Artif. Intell. 293 (2021) 103448.
[3] M. Karaduman, A. Cınar, H. Eren, UAV traffic patrolling via road detection and tracking in anonymous aerial video frames, J. Intell. Rob. Syst. 95 (2019) 675–690.
[4] Z. Zhang, Y. Liu, T. Liu, Z. Lin, S. Wang, DAGN: a real-time UAV remote sensing image vehicle detection framework, IEEE Geosci. Remote Sens. Lett. 17 (11) (2020) 1884–1888.
[5] A. Bouguettaya, H. Zarzour, A. Kechida, A.M. Taberkit, Vehicle detection from UAV imagery with deep learning: a review, IEEE Trans. Neural Netw. Learn. Syst. 33 (11) (2022) 6047–6067.
[6] J. Chai, S. He, H.-S. Shin, A. Tsourdos, Domain-knowledge-aided airborne ground moving targets tracking, Aerosp. Sci. Technol. 144 (2024) 108807.
[7] S. Ren, K. He, R. Girshick, J. Sun, Faster R-CNN: towards real-time object detection with region proposal networks, IEEE Trans. Pattern Anal. Mach. Intell. 39 (6) (2017) 1137–1149.
[8] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: unified, real-time object detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 779–788.
[9] J. Liu, Y. Wei, R. Sun, Y. Yue, LMG-YOLOv10: An efficient defect detection model for aero-engine components based on improved YOLOv10, Aerosp. Sci. Technol. 168 (2026) 110788.
[10] A. Bewley, Z. Ge, L. Ott, F. Ramos, B. Upcroft, Simple online and realtime tracking, in: 2016 IEEE International Conference on Image Processing (ICIP), Ieee, 2016, pp. 3464–3468.
[11] A. Arsenos, E. Petrongonas, O. Filippopoulos, C. Skliros, D. Kollias, S. Kollias, NEFELI: a deep-learning detection and tracking pipeline for enhancing autonomy in advanced air mobility, Aerosp. Sci. Technol. 155 (2024) 109613.
[12] N. Wojke, A. Bewley, D. Paulus, Simple online and realtime tracking with a deep association metric, in: 2017 IEEE International Conference on Image Processing (ICIP), 2017, pp. 3645–3649.
[13] Z. Wang, L. Zheng, Y. Liu, Y. Li, S. Wang, Towards real-time multi-object tracking, in: A. Vedaldi, H. Bischof, T. Brox, J.-M. Frahm (Eds.), Computer Vision – ECCV 2020, Springer International Publishing, Cham, 2020, pp. 107–122.

[14] J. Ma, C. Tang, F. Wu, C. Zhao, J. Zhang, Z. Xu, STCMOT: spatio-temporal cohesion learning for UAV-based multiple object tracking, in: 2024 IEEE International Conference on Multimedia and Expo (ICME), IEEE, 2024, pp. 1–6.
[15] T. Fischer, T.E. Huang, J. Pang, L. Qiu, H. Chen, T. Darrell, F. Yu, Qdtrack: Quasi-dense similarity learning for appearance-only multiple object tracking, IEEE Trans. Pattern Anal. Mach. Intell. 45 (12) (2023) 15380–15393.
[16] Y. Yuan, Y. Wu, L. Zhao, Y. Liu, Y. Pang, TLSH-MOT: drone-view video multiple object tracking via transformer-based locally sensitive hash, IEEE Trans. Geosci. Remote Sens. 63 (2025) 1–16.
[17] S. You, H. Yao, B.-K. Bao, C. Xu, UTM: a unified multiple object tracking model with identity-aware feature enhancement, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 21876–21886.
[18] B. Yang, R. Nevatia, Multi-target tracking by online learning of non-linear motion patterns and robust appearance models, in: 2012 IEEE Conference on Computer Vision and Pattern Recognition, 2012, pp. 1918–1925.
[19] D. Li, C. Hou, D. Yi, Multi-Bernoulli smoother for multi-target tracking, Aerosp. Sci. Technol. 48 (2016) 234–245.
[20] Y. Liu, X. Li, R. Yan, Q. Li, Y. Luo, Stable adaptive Kalman filter with recursive covariance estimation for real-time motion tracking, Aerosp. Sci. Technol. 168 (2026) 110740.
[21] X. Yu, J. Li, J. Xu, Nonlinear filtering in unknown measurement noise and target tracking system by variational Bayesian inference, Aerosp. Sci. Technol. 84 (2019) 37–55.
[22] X. Zhou, V. Koltun, P. Krähenbühl, CenterTrack: tracking objects as points, in: European Conference on Computer Vision, Springer, 2020, pp. 474–490.
[23] C. Feichtenhofer, A. Pinz, A. Zisserman, Detect to track and track to detect, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 3038–3046.
[24] J. Peng, C. Wang, F. Wan, Y. Wu, Y. Fu, Chained-tracker: chaining paired attentive regression results for end-to-end joint multiple-object detection and tracking (2020) 145–161.
[25] J. He, Z. Huang, N. Wang, Z. Zhang, Learnable graph matching: a practical paradigm for data association, IEEE Trans. Pattern Anal. Mach. Intell. 46 (7) (2024) 4880–4895.
[26] Y. Liu, J. She, Q. Yuan, R. Zhou, N. Qi, Real-time small target detection networks for UAV remote sensing, Acta Aeronautica et Astronautica Sinica 45 (14) (2024) 630119.
[27] A. Gu, T. Dao, Mamba: linear-time sequence modeling with selective state spaces, in: First Conference on Language Modeling, 2024.
[28] Y. Liu, R. Zhou, Z. Yao, J. She, N. Qi, STDSD: a spacecraft target detection framework considering similarity and diversity, IEEE Trans. Autom. Sci. Eng. 22 (2025) 11038–11049.
[29] J. Redmon, A. Farhadi, YOLOv3: an incremental improvement, arXiv e-prints (2018).
[30] Z. Liu, Y. Shang, T. Li, G. Chen, Y. Wang, Q. Hu, P. Zhu, Robust multi-drone multi-target tracking to resolve target occlusion: a benchmark, IEEE Trans. Multimed. 25 (2023) 1462–1476.
[31] D. Du, Y. Qi, H. Yu, Y.-F. Yang, K. Duan, G. Li, W. Zhang, Q. Huang, Q. Tian, The unmanned aerial vehicle benchmark: object detection and tracking, in: European Conference on Computer Vision, 2018.
[32] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, H. Ling, Detection and tracking meet drones challenge, IEEE Trans. Pattern Anal. Mach. Intell. 44 (11) (2022) 7380–7399.
[33] R. Kasturi, D. Goldgof, P. Soundararajan, V. Manohar, J. Garofolo, R. Bowers, M. Boonstra, V. Korzhova, J. Zhang, Framework for performance evaluation of face, text, and vehicle detection and tracking in video: data, metrics, and protocol, IEEE Trans. Pattern Anal. Mach. Intell. 31 (2) (2009) 319–336.
[34] E. Ristani, F. Solera, R. Zou, R. Cucchiara, C. Tomasi, Performance measures and a data set for multi-target, multi-camera tracking, in: G. Hua, H. Jégou (Eds.), Computer Vision – ECCV 2016 Workshops, Springer International Publishing, Cham, 2016, pp. 17–35.
[35] Luiten, JonathonOsep, AljosaDendorfer, PatrickTorr, PhilipGeiger, AndreasLeal-Taixe, LauraLeibe, Bastian, HOTA: a higher order metric for evaluating multi-object tracking, Int. J. Comput. Vis. 129 (2) (2021).
[36] Y. LIU, J. SHE, Y. SONG, Z. LIU, X.U. Huarui, Q. YUAN, Q.I. Naiming, A unified framework for small object analysis in UAV imagery: tracking single target from multiple detected targets, Chin. J. Aeronaut. (2025) 103917.
[37] G. Ciaparrone, F. Luque Sánchez, S. Tabik, L. Troiano, R. Tagliaferri, F. Herrera, Deep learning in video multi-object tracking: a survey, Neurocomputing 381 (2020) 61–88.
[38] N. Aharon, R. Orfaig, B.-Z. Bobrovsky, BoT-SORT: robust associations multi-pedestrian tracking,(2022), arXiv preprint arXiv:2206.14651 .
[39] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, H. Meng, Strongsort: make deepsort great again, IEEE Trans. Multimedia 25 (2023) 8725–8737.
[40] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang, Bytetrack: multi-object tracking by associating every detection box, in: European Conference on Computer Vision, Springer, 2022, pp. 1–21.
[41] W. Lv, Y. Huang, N. Zhang, R.-S. Lin, M. Han, D. Zeng, DiffMOT: a real-time diffusion-based multiple object tracker with non-linear prediction, in: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 19321–19330.
[42] F. Xie, Z. Wang, C. Ma, Diffusiontrack: point set diffusion model for visual object tracking, in: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 19113–19124.
[43] M. Yao, J. Peng, Q. He, B. Peng, H. Chen, M. Chi, C. Liu, J.A. Benediktsson, MM-Tracker: motion Mamba with margin loss for UAV-platform multiple object tracking,(2024), arXiv preprint arXiv:2407.10485 .